

Below is a **very concrete, practical example** taken from each Android layer.

1. Application layer

Example: *Camera app*

➔ When you open the Camera application and press the shutter button.

What really happens at this layer?

The Camera app (APK) calls high-level Android APIs to start preview and capture a photo.

2. Application Framework layer

Example: `CameraManager` system service

➔ The Camera app uses the **Camera API** provided by the framework:

`CameraManager`

Practical role:

The framework exposes services such as:

- Camera service
- Activity manager
- Window manager

The app never talks directly to the hardware.

It talks to **framework services**.

3. Android Runtime (ART) + core libraries

Example:

Executing this line inside the Camera app:

```
new Thread(...)
```

or

```
ArrayList<Bitmap>
```

Practical role:

- The app code runs inside **ART**
- Java/Kotlin core libraries are used (collections, threads, I/O, etc.)

- Bytecode is executed and optimized by the runtime

So:

➔ your app logic is executed here.

4. Native Libraries layer

Example:

Using a native image encoder (JPEG / PNG) from the media stack.

➔ When the photo is saved, Android internally uses native C/C++ libraries such as:

- media codecs
- image encoders

Practical role:

Heavy operations (image processing, audio/video encoding, graphics rendering) are done using **native libraries** for performance.

5. Hardware Abstraction Layer (HAL)

Example:

Camera HAL implementation

➔ The framework calls the **camera HAL** to talk to the real camera device.

Practical role:

HAL translates:

“Take a picture”
into
vendor-specific hardware commands

So the same Android framework can work on different camera hardware.

6. Linux Kernel layer

Example:

Camera device driver and memory management

➔ The camera HAL finally interacts with:

- the camera driver
- kernel memory

- scheduling and device I/O

Practical role:

The kernel actually:

- accesses the hardware
 - manages buffers
 - handles interrupts from the camera sensor
-

One real-life chain (very clear)

When you take a photo:

Camera App (Application layer)

- calls **Camera API (Framework)**
 - executed by **ART + libraries (Runtime)**
 - uses **native media/image libraries**
 - talks to **Camera HAL**
 - accesses the device through **Linux kernel driver**
-

In one short table

Layer	Practical example
Application	Camera app
Application Framework	CameraManager / camera service
Android Runtime	Execution of app Java/Kotlin code (e.g., Thread, collections)
Native Libraries	Native image / media encoding libraries
HAL	Camera HAL implementation
Linux Kernel	Camera driver and device access

This is a **real and accurate layered interaction**.

Here is a **second, different real-world example** for *each* Android layer, using a **location (GPS) request** instead of the camera.

Scenario

You open a navigation app and request your current location.

1. Application layer

Example: A navigation or maps application.

➔ The user presses “**My location**”.

At this layer:

The application simply asks Android for the device position.

2. Application Framework layer

Example: `LocationManager` (or the fused location provider service)

➔ The app calls:

```
requestLocationUpdates(...)
```

At this layer:

The framework service:

- handles permissions,
- selects the proper provider (GPS, network, etc.),
- manages callbacks to the app.

The application still has no idea how the hardware works.

3. Android Runtime (ART) + core libraries

Example:

The app code that handles the location callback:

```
onLocationChanged(Location location)
```

At this layer:

- The Java/Kotlin code of the app is executed by ART
- Core libraries handle objects, threads, and callbacks

So the listener logic and data processing run here.

4. Native Libraries layer

Example:

Native GNSS (GPS) support libraries used internally by the system.

➔ Position calculation and low-level parsing of GNSS data are done in native C/C++ components for performance.

At this layer:

The framework relies on native code to efficiently process satellite data.

5. Hardware Abstraction Layer (HAL)

Example:

GNSS (GPS) HAL implementation.

➔ The framework asks the **GNSS HAL** for positioning data.

At this layer:

The HAL converts standard Android GNSS requests into vendor-specific commands for the GPS chip.

This is what allows the same Android framework to work on different GPS hardware.

6. Linux Kernel layer

Example:

GNSS device driver and serial / bus driver.

➔ The HAL finally communicates with the GPS chip through:

- kernel drivers
- kernel I/O and interrupts

At this layer:

The kernel:

- receives raw data from the GNSS hardware,

- manages buffers,
 - provides access to upper layers.
-

Full chain (second example)

When you request your position:

Navigation app (Application)

- **LocationManager service (Framework)**
 - **App callback executed by ART (Runtime)**
 - **Native GNSS libraries (Native libraries)**
 - **GNSS HAL**
 - **Linux kernel driver → GPS chip**
-

✓ Short table (second example)

Layer	Practical example (location use case)
Application	Navigation / maps app requesting location
Application Framework	<code>LocationManager</code> / fused location service
Android Runtime	Execution of <code>onLocationChanged()</code> and app logic
Native Libraries	Native GNSS processing libraries
HAL	GNSS (GPS) HAL implementation
Linux Kernel	GNSS device driver and hardware access

This gives you a **second concrete and realistic layered example**, complementary to the camera example.